HYPERION AI

SYSTEM DESIGN DOCUMENT

Production Claude Agents with MCP + CRM

Reference architecture for stateful chatbots that read and write CRM safely

PREPARED FOR

Firms extending Claude into CRM-integrated chatbot capabilities

Vitalijus Alsauskas

Software and AI Engineer

Production Claude Agents with MCP + CRM - System Design

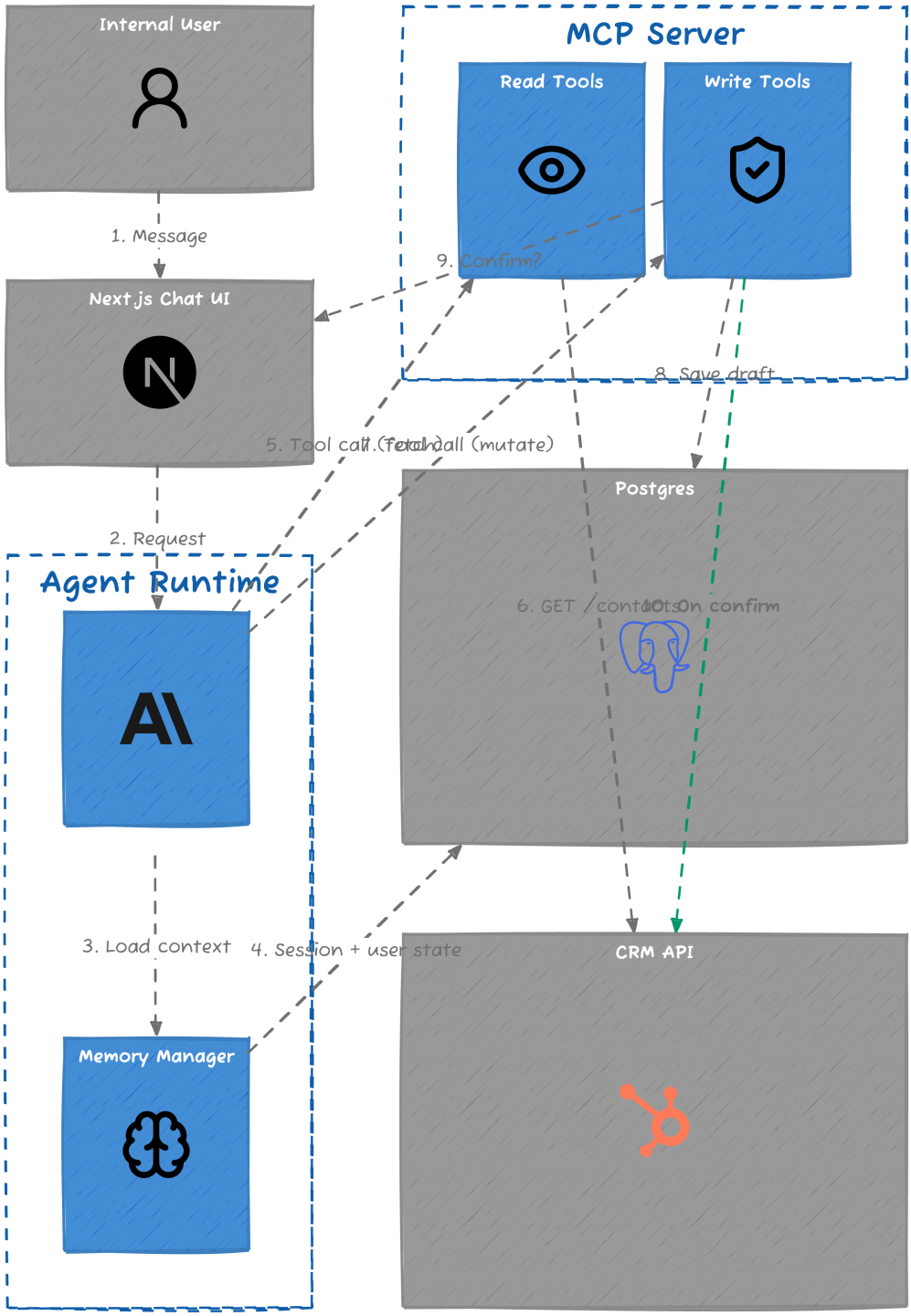
Problem Framing

Claude plus your CRM usually breaks at the same place: the chatbot has memory within a single conversation but forgets everything outside of it. So users end up re-explaining context every session. And when Claude does try to write to the CRM, you either trust it blindly (bad) or add so many guardrails it becomes useless (also bad).

This reference shows the pattern that works in production: MCP tools give the agent a proper interface to your CRM, three layers of memory so users don't repeat themselves, and confirmation-gated writes so the agent drafts but never silently changes records.

Written for teams extending an existing Claude setup with CRM-aware chatbot capabilities.

System Diagram



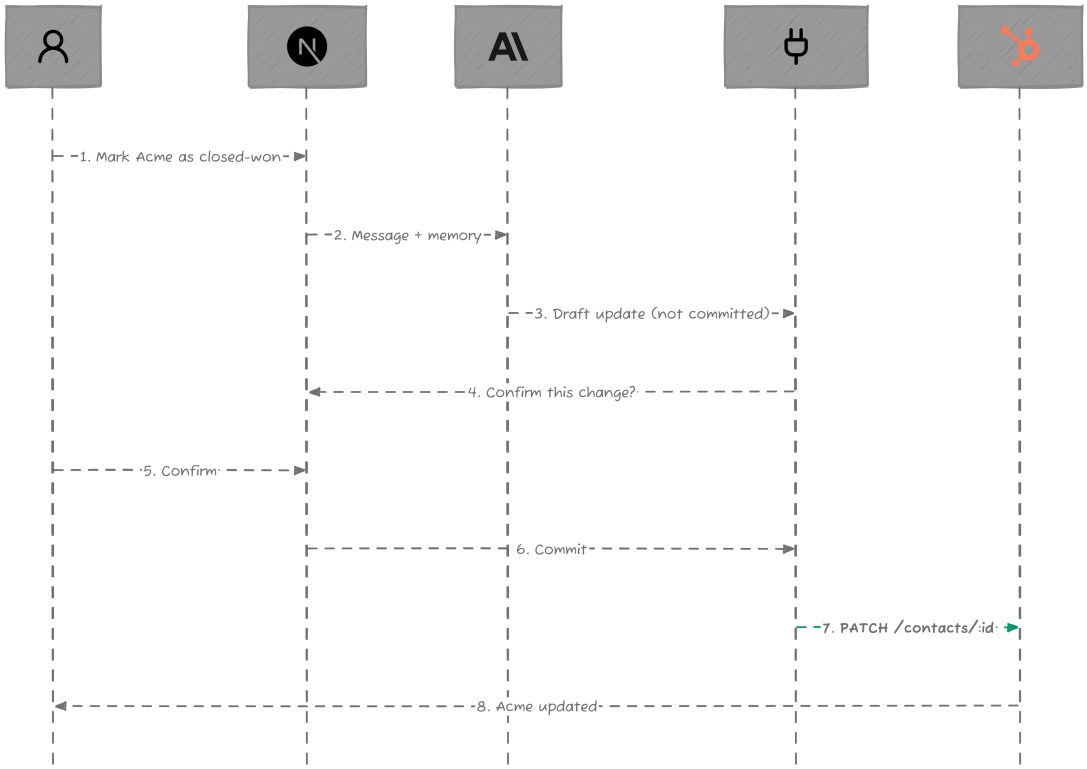
Source: `docs/architecture.d2`

Every CRM capability the agent needs is exposed as an MCP tool. Read tools flow directly. Write tools route through a guardrail that creates a draft, asks the user to confirm, and only commits on approval. Every tool call is audit-logged.

Tool Choices

LAYER	PICK	WHY THIS	ALTERNATIVES CONSIDERED
Chat UI	Next.js + React streaming	Server Components for auth, streaming for response UX	Plain React SPA (weaker streaming support), Streamlit (not production-ready)
Agent Runtime	FastAPI + Anthropic SDK	Async-native, strong typing, Python ecosystem alignment with Claude SDK	Node + `@anthropic-ai/sdk` (fine, Python preferred here), direct LangChain (too much abstraction)
Tool Layer	Model Context Protocol (MCP)	Clean contract, swap CRMs without agent changes, growing standard	Custom function calling (works but CRM lock-in), LangChain tools (heavier, harder to debug)
Session memory	Postgres rows (conversation history)	Durable, structured, queryable by user + time	Redis only (lost on TTL), in-process (lost on restart)
Long-term memory	Vector store (pgvector)	Semantic recall of past conversations, co-located with Postgres	Pinecone (extra service, extra cost), Weaviate (overkill)
Audit log	Immutable S3 bucket + Postgres mirror	Append-only, compliance-grade, queryable	Regular DB table (mutable), CloudTrail only (misses app-level context)
CRM integration	HubSpot / Salesforce / Pipedrive via official APIs	Official, documented, rate-limited in known ways	Scraping (fragile), Zapier middleware (extra hop)

Data Flow



Source: `docs/flow.d2`

Read flow

1. User message hits the chat UI, streams to agent
2. Agent loads three memory layers: session (last N turns), user (preferences, last interactions), account (company-wide context)
3. Agent sends prompt + MCP tool schema to Claude
4. Claude decides which tools to call (can chain multiple tool calls in sequence)
5. Each tool call hits the MCP server, which calls the CRM API, logs to audit, returns
6. Claude composes final answer, agent streams back to UI
7. Conversation turn persists to Postgres + relevant facts embed into vector store

Write flow (confirmation-gated)

1. User asks for a mutating action ("close Acme, mark as closed-won")
2. Claude returns a tool call for the mutating tool (e.g., `update_stage`)
3. Agent detects mutation, routes through guardrail

4. Guardrail creates a draft row in Postgres with proposed change
5. Agent returns "Draft: change Acme stage to closed-won. Confirm?" to UI
6. User clicks confirm, UI sends commit request
7. Agent fires the actual CRM API write, logs commit to audit with before/after
8. User sees confirmation message

Error paths

- **Claude timeout**: retry with exponential backoff, fallback to Claude Haiku after 2 failures
- **MCP tool error**: bubble up as a tool result with error context, agent can decide to retry or inform user
- **CRM API error**: tool result is error, agent retries or tells user "I couldn't reach the CRM, try again"
- **User rejects draft**: draft row marked rejected, nothing commits to CRM
- **Memory load failure**: fall back to empty memory, log warning - conversation proceeds, quality just lower

Component Breakdown

- **Chat UI** (Next.js 15, Server Components, streaming SSE): message input, streaming output, confirmation drawer for drafts, audit view per user
- **Agent Orchestrator** (FastAPI + Anthropic SDK): tool-use loop, prompt composition, memory integration
- **Memory Manager** (Python + SQLAlchemy + pgvector): three-layer memory retrieval + persistence
- **MCP Server** (Python MCP SDK): tool registration, CRM adapter per backend (hubspot.py, salesforce.py, pipedrive.py)
- **Write Guardrails** (agent middleware): intercept mutating tool calls, draft persistence, confirmation flow
- **Audit Logger** (FastAPI middleware + S3 writer): every tool call + commit logged with full context
- **Admin UI** (Next.js route under /admin): view audit trail, search conversations, revoke user access

Implementation Phases

Phase 1: Foundation (week 1-2)

Ships: Read-only chatbot that can answer questions using CRM data, with session memory.

- Chat UI with streaming
- Agent orchestrator + MCP server with 3 read tools (get_contact, search_contacts, get_history)
- Session memory via Postgres
- One CRM backend wired up (whichever you're on)
- Audit log for every tool call

Phase 2: Memory + Writes (week 3-4)

Ships: Full memory layers + confirmation-gated write support.

- User + account memory layers (not just session)
- Vector store for long-term semantic recall
- Draft persistence + confirmation flow for write tools
- One write tool enabled end-to-end (log_activity or update_stage)
- Admin UI for audit review

Phase 3: Production Polish (week 5-6)

Ships: Additional CRM backends if needed, observability, deploy.

- Second CRM backend (if the account has more than one)
- Sentry error tracking wired in
- Runbook: what to do when agent hallucinates, how to replay conversations, how to revoke user access
- Knowledge transfer session

Risk & Mitigation

1. **Risk:** Agent writes to CRM silently, user finds surprise records. **Mitigation:** Confirmation-gated write flow. Every mutating tool goes through the guardrail. No writes commit without explicit user approval. Drafts persist in DB so nothing is lost between draft and commit.
2. **Risk:** Agent hallucinates CRM data instead of calling the tool. **Mitigation:** System prompt explicitly forbids answering about CRM data without a tool call. Tool schemas strict about required fields. Audit log shows tool usage - if hallucinations slip through, you see them in review.
3. **Risk:** PII in conversation history violates compliance. **Mitigation:** Anthropic zero-retention flag. Memory layer redacts sensitive fields before embedding into vector store. User-level data isolation - one user cannot see another's memory.
4. **Risk:** CRM API rate limits during heavy usage. **Mitigation:** MCP server caches recent reads (TTL 60s), batches where possible, exponential backoff on 429. Hard concurrency limit per user to prevent runaway tool calls.
5. **Risk:** Audit log tampering or loss. **Mitigation:** Immutable S3 bucket with Object Lock. Postgres mirror for fast query. Cannot delete or modify existing audit rows via app code.

What I Need From You

- **Which CRM** you're on (HubSpot, Salesforce, Pipedrive, other?)
- **Which CRM actions** the chatbot should cover in Phase 1 (top 5 reads + top 2 writes)
- **Sample user scenarios** - 3-5 example conversations you want the bot to handle
- **Current Claude setup** - are you using the API directly, or via a vendor like Anthropic + your own wrapper? Any existing MCP work?
- **Compliance requirements** - any data residency, retention, or redaction rules I should know about?
- **30-minute kickoff call** to walk through this doc against your actual setup

---

This document is yours to keep regardless of whether we end up working together. The MCP + confirmation pattern here will outlive any specific CRM integration.